Gas Type Classification Using Machine Learning: A Comparative Machine Learning Study

Adan Delgado

Eastern New Mexico University

CSCI 460 – Introduction to Applied Machine Learning

Author E-mail: Adan.Delgado@enmu.edu

May 4, 2026

Abstract

Chemical gas sensors degrade over time. This is a phenomenon known as “sensor drift”, which makes reliable gas identification difficult without occasional recalibration. This project aims to apply machine learning to the Gas Sensor Array Drift Dataset, obtained from the UCI Machine Learning Repository, to classify six distinct gases based on sensor readings collected over a 36-month period. The dataset contains 13,910 measurements generated by an array of 16 metal-oxide sensors exposed to the gases: Ethanol, Ethylene, Ammonia, Acetaldehyde, Acetone and Toluene. They were exposed at varying levels of concentration, and each measurement was described by 128 engineered features. The goal of this project is to identify which machine learning model best classifies gas type from sensor data, and to compare the relative performance of six different algorithms. Prior to modeling, exploratory data analysis (EDA) was conducted to examine: class balance, feature distributions, correlations, missing values and the presence of outliers. Preprocessing included: StandardScaler normalization and Principal Component Analysis for dimensionality reduction. Six models were trained and tuned using 5-fold cross-validated Grid Search: K-Nearest Neighbors, Decision Tree, Support Vector Machine (SVM), Logistic Regression, Random Forest, and XGBoost. All tuned models were evaluated on a held out test set using accuracy, weighted F-1 score, precision, and recall. KNN achieved the highest test accuracy at 99.32%, followed closely by Random Forest at 99.14% and XGBoost at 99.07%. These results indicate that this dataset is easy to separate and is well suited for classification, with distance based and ensemble methods performing particularly well.

1. Introduction

1.1 Problem Definition

Gas sensors are used widely in industrial safety monitoring systems, environmental sensing and even medical diagnostics. “Since 1962 it has been known that absorption or desorption of a gas on the surface of a metal oxide changes the conductivity of the material, this phenomenon being first demonstrated using zinc oxide thin film layers” (Fine et al., 2010). In other words, metal-oxide detectors work by measuring the changes in electrical resistance when a gas molecule interacts with the metal oxide surface. However, metal-oxide sensors degrade over time due to a process known as “sensor drift”, in which a sensor's chemical response gradually degrades from its original calibrated accuracy. This degradation can cause a sensor that once reliably identified a given gas to produce increasingly inaccurate readings. This reduces the overall reliability of the detection system.

The task addressed in this project is a multi-class classification problem: given a 128-feature vector of sensor readings, predict which of the six gases: Ethanol (class 1), Ethylene (class 2), Ammonia (class 3), Acetaldehyde (class 4), Acetone (class 5), or Toluene (class 6) produced the reading. While the broader research context around this dataset focuses on

compensating for drift over time, this project focuses on the classification task itself, evaluating how well several machine learning algorithms can distinguish between gas types using the full 36-month collection of sensor data.

1.2 Dataset Description

The Gas Sensor Array Drift Dataset was obtained from the UCI Machine Learning Repository (ID:224). It was collected between January 2007 and February 2011 at the ChemoSignals Laboratory, BioCircuits Institute, University of California San Diego. The dataset contains 13,910 measurements recorded from an array of 16 metal-oxide gas sensors. The sensors were exposed to six pure gaseous substances at concentrations ranging from 5 to 1000 parts per million by volume (ppmv). It is worth noting that the documentation states: "Batch10.dat was updated on 10/14/2013 to correct some corrupted values in the last 120 lines of the file." According to the UCI dataset page.

Each measurement is represented by a 128-dimensional feature vector, comprising 8 engineered features extracted from each of the 16 sensors. These 8 features per sensor consist of 2 steady state features and 6 transient features derived from exponential moving averages applied to the rising and decaying portions of the sensor response. The target variable is a class label from 1 to 6 indicating which gas produced the measurement. (Vergara, 2012)

1.3 Objectives

This project aims to answer the following research questions:

1. Which machine learning algorithm achieves the highest classification accuracy on the Gas Sensor Array Drift Dataset?
2. How do simpler models such as Logistic Regression and Decision Trees compare to ensemble methods such as Random Forest and XGBoost on this dataset?
3. Does dimensionality reduction via PCA meaningfully reduce the feature space while preserving the classification performance?

1.4 Report Structure

The remainder of this report is organized as follows. Section 2 presents the exploratory data analysis conducted prior to modeling. Section 3 details the preprocessing pipeline and methodology, including model selection, hyperparameter tuning strategy and evaluation metrics. Section 4 presents and discusses the results of all six tuned models. Section 5 concludes the report with a summary of findings and directions for future work.

2. Exploratory Data Analysis

2.1 Missing Values

The first step of the EDA was to check for missing values. A complete scan of the feature matrix revealed zero missing values across all 128 features and 13,910 samples. This is consistent with the UCI dataset documentation, which explicitly made note that no missing values are present. No imputation was necessary.

2.2 Class Distribution

Figure 1 shows the distribution of samples across the six gas classes. The class counts are as follows:

- Ethanol (2,565)
- Ethylene (2,926)
- Ammonia (1,641)
- Acetaldehyde (1,936)
- Acetone (3,009)
- Toluene (1,833)

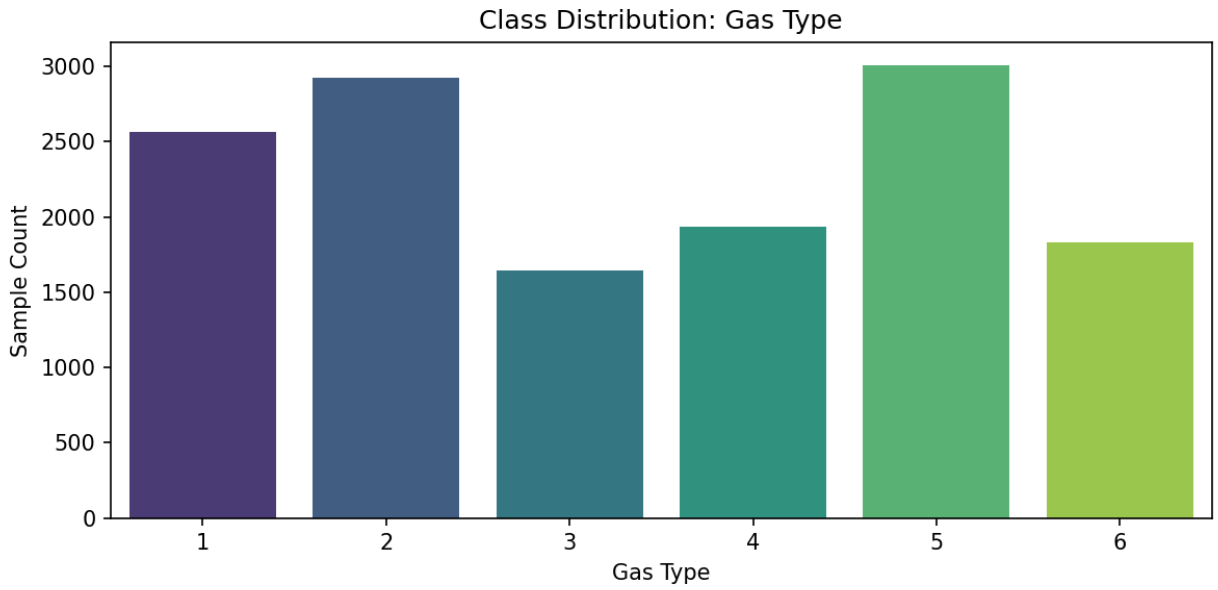

Figure 1: Class Distribution

The dataset is mildly imbalanced, which is worth noting. Acetone contains nearly twice as many samples as Ammonia. However, the imbalance does not appear to be severe enough to warrant resampling techniques. It does suggest that weighted evaluation metrics such as weighted F1-

scores are more appropriate than raw accuracy alone, since accuracy could be inflated by the majority class.

2.3 Feature Distribution

Figure 2 shows the distributions of the first six features as a representative sample. Features 0 through 4 all exhibit right-skewed distributions, with most values concentrated near zero and long tails extending towards higher values. This pattern is consistent with the sensor resistance data, where most readings are small and occasional large spikes occur at higher gas concentrations. Feature_1 is particularly notable, because it shows an extremely sharp spike at near zero values with almost no spread. This suggests very low variance across samples. Feature_5 on the other hand exhibits a left-skewed distribution with values extending into negative territory. This is consistent with it being one of the EMA decaying portion features described in the UCI documentation, which can produce negative scalar values. Though this is mostly an inference based on the feature ordering described in the dataset documentation, not a directly confirmed fact.

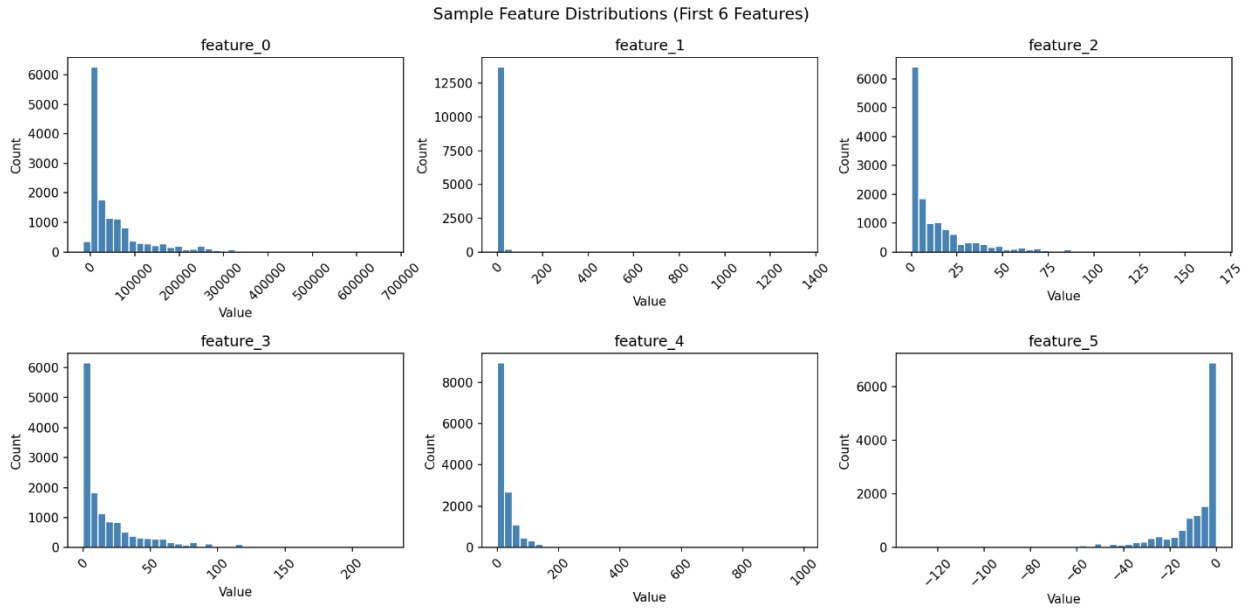

Figure 2: Sample Feature Distributions

2.4 Feature Correlations

Figure 3 shows a correlation heatmap for the first 20 features. Distinct clusters of strong positive and negative correlations are clearly visible. This is expected given the dataset structure. The 128 features are organized as 8 features per sensor across 16 sensors, meaning features derived from the same sensor are likely to be highly correlated with one another. The presence of this redundancy is a key justification for applying PCA prior to modeling, since it allows the

correlated features to be compressed into a smaller set of uncorrelated components without significant loss of information.

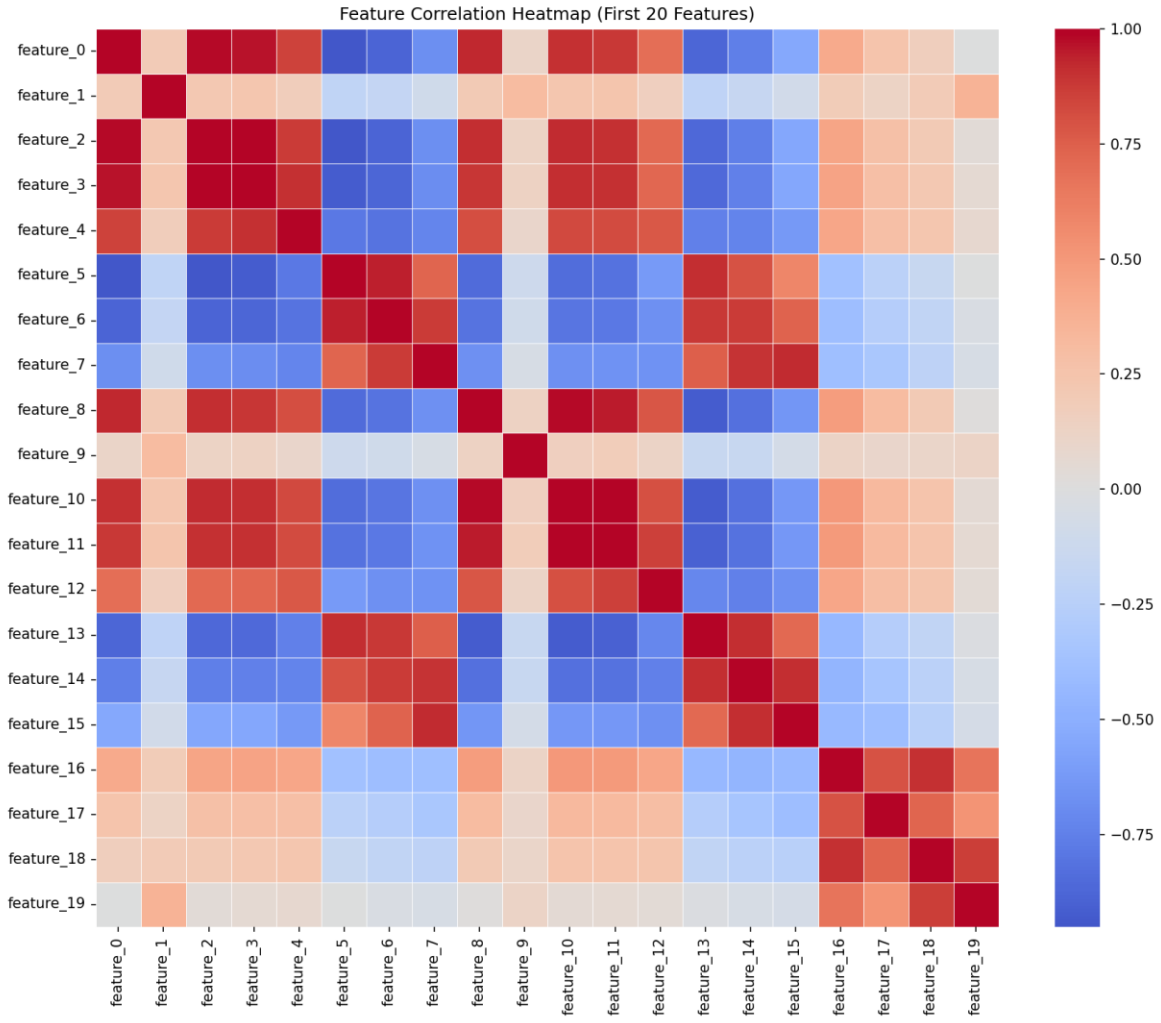

Figure 3: Feature Correlation Heatmap

2.5 Class Separation

Figure 4 shows box plots of the first four features grouped by gas class, with individual outlier points hidden for visual clarity. There are many outliers, and this will be discussed in the next section. Fairly visible differences in the median and interquartile range across gas classes are present in all four features, with some classes showing notably higher median values than others. For example, feature_0, class 1 and class 5 show substantially higher median values compared to classes 2, 3, and 6. These differences suggest that the features carry meaningful discriminative information and that the classes should be reasonably separable by a well-trained

classifier.

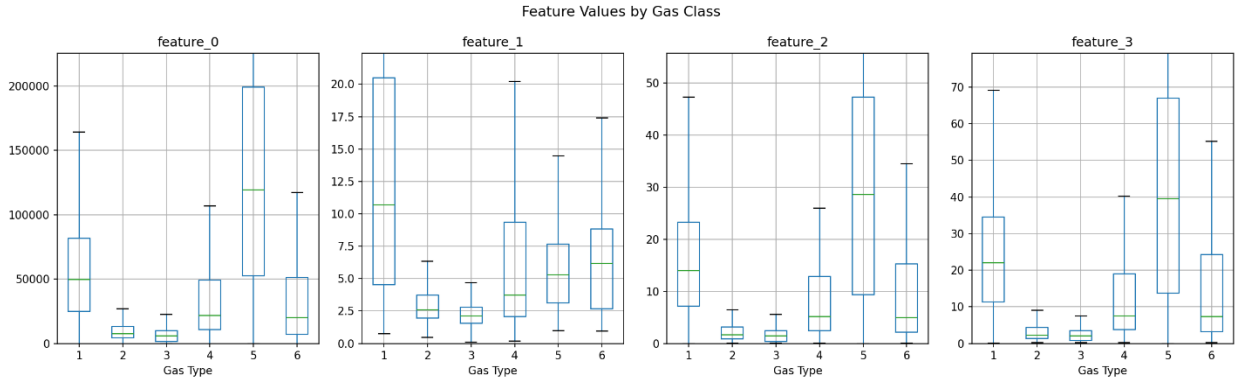

Figure 4: Feature Values By Class

2.6 Outlier Detection

Outliers were identified using the basic Interquartile Range (IQR) method, flagging any values falling below Q1 – 1.5 * IQR or above Q3 + 1.5 * IQR. A total of 80,397 outlier values were detected across all 128 features, spread across all features rather than concentrated in only a few. As shown in Figure 5, the distribution of outliers is consistent across the feature space with no single feature particularly standing out as peculiar. Importantly, given the class separation visible in Figure 4, many of these flagged values likely correspond to genuine differences in sensor responses between gas classes rather than data errors — certain classes consistently produce higher feature values that fall outside the IQR of the overall distribution. For this reason, no outliers were removed. Removing them would risk discarding real signal rather than plain noise.

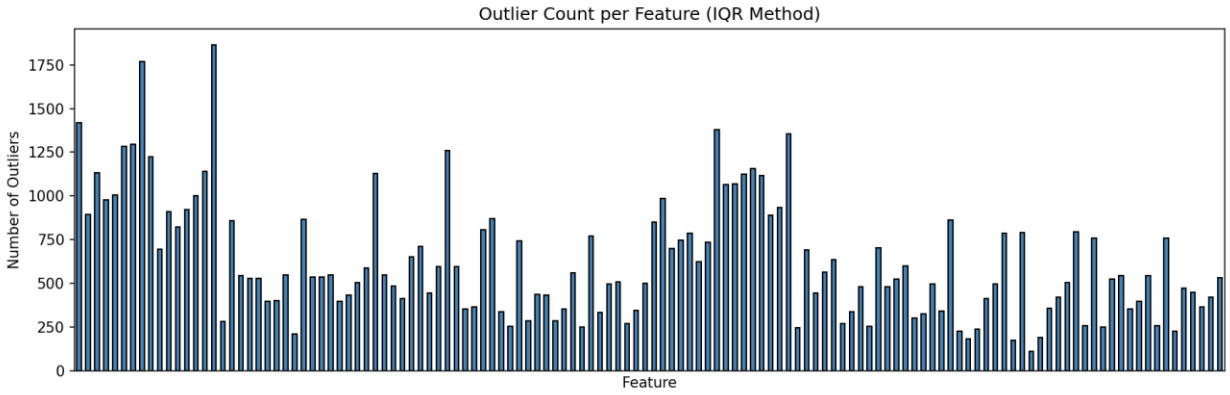

Figure 5: Outliers

3. Methodology

3.1 Data Preprocessing

Prior to modeling, the raw data required two key preprocessing steps: feature scaling and dimensionality reduction. No missing value imputation was necessary, as established in section 2.

Feature Scaling

All 128 features were standardized using scikit-learn's StandardScaler, which transforms each feature to have a mean of zero and a standard deviation of one. This step is necessary because several of the models used in this project (particularly KNN and SVM) are sensitive to the scale of input features. KNN relies on distance calculations between samples. This means a feature with values in the hundreds of thousands would dominate the distance calculation over a feature with near zero, regardless of how informative each feature actually is. SVM similarly constructs decision boundaries based on distances and performs poorly on unscaled data. Logistic Regression also benefits from scaling for numerical stability during optimization steps.

Critically, the scaler was fitted exclusively on the training set and then applied to the test set using transform only. Fitting the scaler on the full dataset before splitting would have constituted a data leakage. In other words, the model would have indirect knowledge of the test set's distribution during the training, artificially inflating performance estimates.

Handling Outliers

As discussed in section 2.6, the 80,397 outlier values detected by the IQR method were not removed. These values are consistent with genuine differences in sensor responses between gas classes rather than data entry errors and removing them would risk discarding real discriminative signals.

3.2 Feature Engineering and Dimensionality Reduction

Principal Component Analysis (PCA) was applied to the scaled training data to reduce the 128-dimensional feature space to a smaller and more manageable set of uncorrelated components. PCA works by identifying the directions of maximum variance in the data and projecting the data onto those directions. Because highly correlated features contain redundant information (like those seen in the heatmap from section 2.4), PCA can represent that information more compactly without meaningful loss.

The number of components was not set manually. Instead, n_components = 0.95 was passed to scikit-learn's PCA implementation, which automatically selects the minimum number of components needed to explain 95% of the total variance. As shown in the scree plot (Figure 6), this threshold was reached at 12 components. A reduction from 128 features down to 12, retaining 95% of the variance. This aggressive dimensionality reduction is possible precisely

because of the high inter-feature correlation identified during EDA. The same transform only rule applied to the test set here as with scaling. PCA was fitted on the training set only and applied to the test set without refitting.

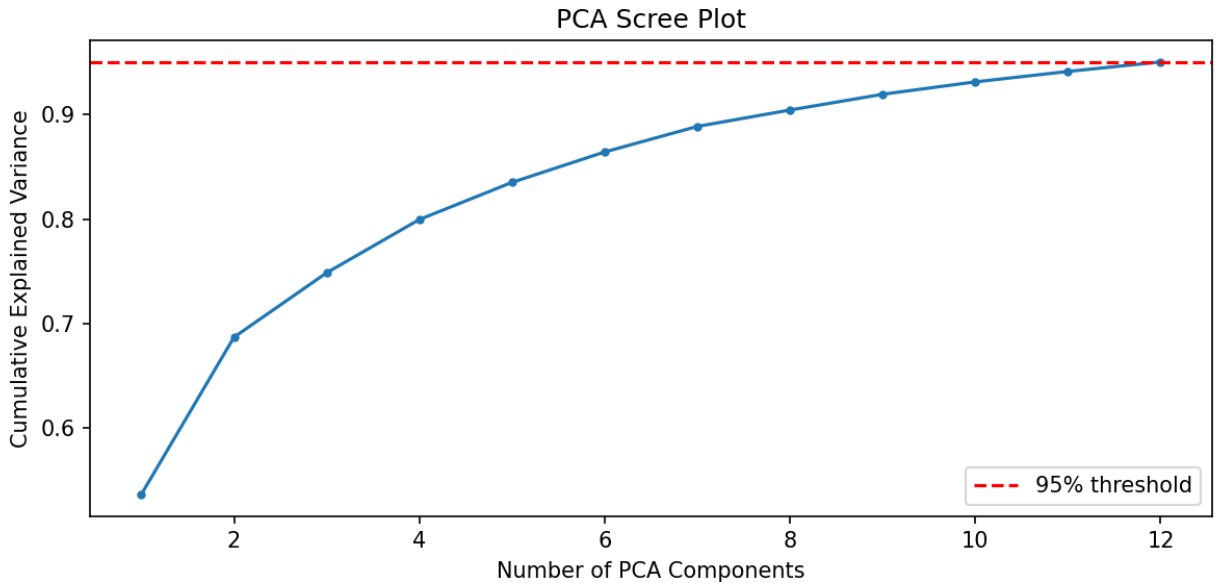

Figure 6: PCA Scree Plot

3.3 Machine Learning Models

Six machine learning models were selected for this project. The selection was informed by a LazyPredict benchmark survey, which quickly evaluated the accuracy of many classifiers on the dataset prior to formal tuning. As shown in Figure 7, the top performing classifiers (including ExtraTreesClassifier, LGBMClassifier, RandomForestClassifier, KNeighborsClassifier, and XGBClassifier) all achieved accuracies at or above 0.99, while simpler linear models such as RidgeClassifier and GaussianNB performed considerably lower. This informed the decision to prioritize ensemble and distance-based methods in the final model selection. The six models chosen represent a range of different algorithms, from simple baselines to powerful ensemble methods.

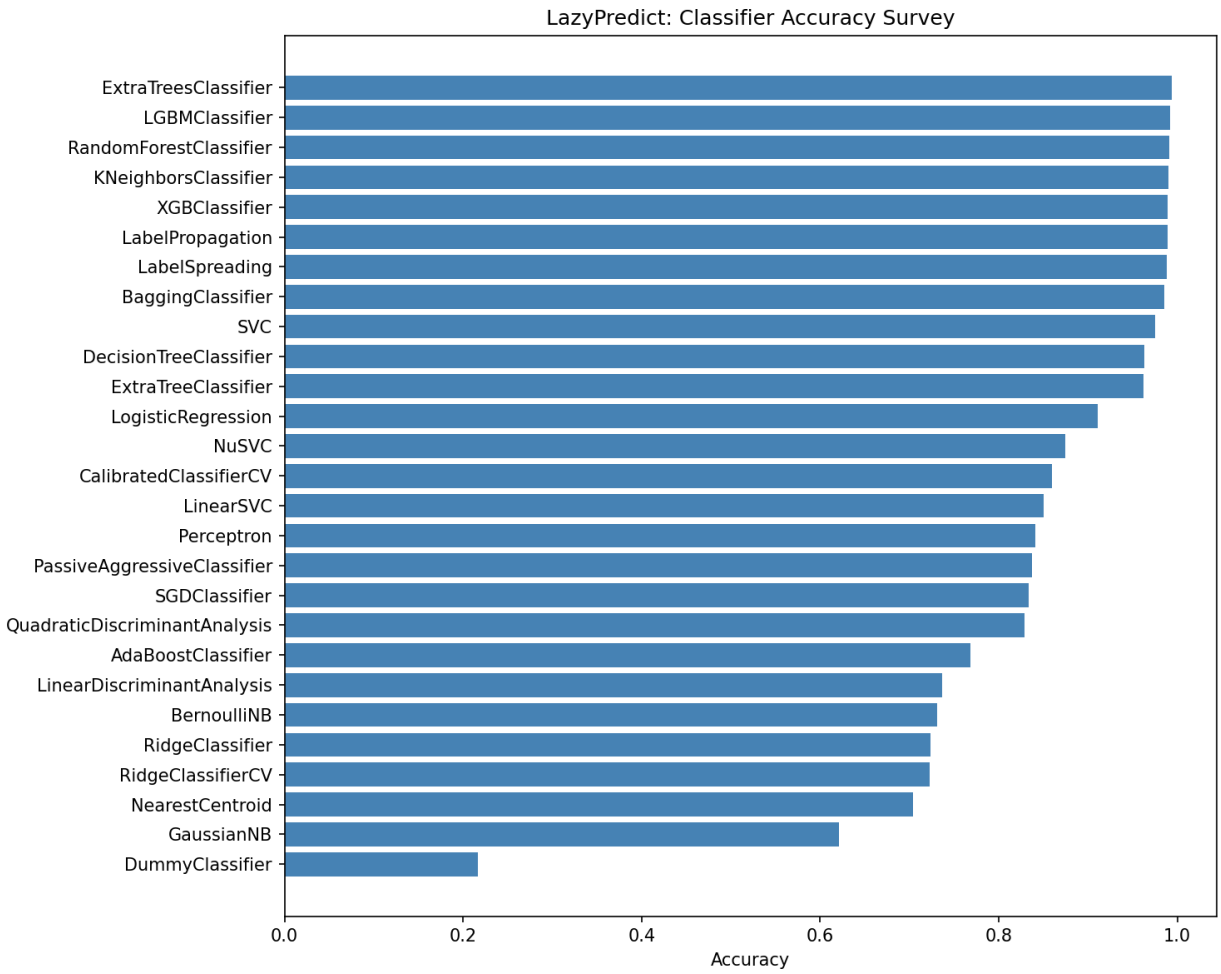

Figure 7: LazyPredict Classifier Accuracy Survey

K-Nearest Neighbors (KNN)

KNN classifies a sample by identifying the K closest training samples in feature space and assigning the majority class among those neighbors. It makes no assumptions about the underlying data distribution, which makes it flexible. It was included because it performed strongly in the LazyPredict survey and represents a non-parametric, distance based approach that contrasts well with other models.

Decision Tree

A Decision Tree recursively partitions the feature space by selecting the feature and threshold that best separates the classes at each node according to a criterion such as Gini or entropy. It is highly interpretable but prone to overfitting on training data without constraints on tree depth. It was included as a single model baseline that can be compared directly against its ensemble counterpart, Random Forest.

Support Vector Machine (SVM)

An SVM finds the hyperplane that maximizes the margin between classes in the transformed feature space. Using RBF kernel, it can construct non-linear decision boundaries. SVMs are well suited to high-dimensional data and benefit significantly from feature scaling. It was included as a strong, non-ensemble classifier.

Logistic Regression

Logistic Regression models the probability of class membership using a linear combination of features passed through a softmax function for multi-class problems. It is the simplest model in the lineup and serves as a linear baseline. Its relatively lower performance provides a useful contrast that illustrates the limitations of linear decision boundaries on this dataset.

Random Forest (*Required)*

Random Forest is an ensemble method that trains many decision trees on random subsets of the training data and features, then aggregates their predictions by majority vote. This process is known as "bagging" and it reduces the variance and overfitting tendency of individual trees. It was required by the project specification and represents the bagging family of ensemble methods.

XGBoost (*Required)*

XGBoost is a gradient boosting method that trains trees sequentially, with each new tree correcting the errors of the previous ones. Unlike bagging, boosting focuses the model's attention on difficult samples over time. It was required by the project specifications and represents the boosting family of ensemble methods.

3.4 Experimental Setup

The dataset was split into 80% training (11,128 samples) and 20% test (2,782 samples) using a stratified split, ensuring that the class proportions were preserved in both sets. Stratification was used because the dataset is mildly imbalanced, as noted in section 2.2. The test set was held out entirely and only used for final evaluation. No model selection or tuning decisions were made using test set performance.

All experiments were conducted in Python. The core libraries used were scikit-learn (Pedregosa et al., 2011) for preprocessing, model training, and evaluation, XGBoost (Chen & Guestrin, 2016) for the XGBoost classifier, pandas (McKinney, 2010) and numpy (Harris et al., 2020) for data manipulation, and LazyPredict for the preliminary model survey. A fixed random seed of 42 was used throughout to ensure reproducibility.

3.5 Hyperparameter Tuning

Each model was tuned using GridSearchCV with 5-fold stratified cross-validation, optimizing for weighted F1-scores. For each fold, the model was trained on 80% of the training data and validated on the remaining 20%, rotating through all five folds. The best hyperparameter combination was selected based on the average weighted F1-score across all five folds.

The hyperparameter grids explored for each model were as follows:

- KNN: number of neighbors (3, 5, 7, 11), weight function (uniform, distance), distance metric (Euclidean, Manhattan).
- Decision Tree: maximum depth (5, 10, 20, None), minimum sample to split a node (2, 5, 10), splitting criterion (gini, entropy).
- SVM: regularization parameter C (1,10), kernel (rbf only), gamma (scale only).
- Logistic Regression: regularization parameter C(0.01, 0.1, 1, 10), solver (lbfgs, saga), maximum iterations fixed at 500.
- Random Forest: number of estimators (100, 200, 300), maximum depth (10, 20, None), minimum samples to split (2,5).
- XGBoost: number of estimators (100, 200), maximum depth (3, 6, 9), learning rate (0.05, 0.1, 0.2).

The search ranges were chosen based on established conventions for each model type. For KNN, neighbor counts of 3, 5, 7, and 11 were chosen to cover both fine-grained and smoother decision boundaries, while both distance metrics and weight functions were included to account for the varying feature scales present before scaling. For the Decision Tree, depth limits of 5, 10, 20, and None were chosen to explore the full spectrum from heavily pruned to fully grown trees, directly targeting the overfitting tendency identified as a concern for single trees. For SVM, only the RBF kernel was included because the LazyPredict survey and EDA both indicated non-linear class boundaries, making a linear kernel unlikely to be competitive. The C values of 1 and 10 cover a moderate range of margin softness without venturing into extreme regularization. For Logistic Regression, the C range of 0.01 to 10 spans three orders of magnitude to test both strongly and weakly regularized linear models, while the saga solver was included alongside lbfgs because it handles larger datasets more efficiently. For Random Forest and XGBoost, the estimator counts and depth ranges were chosen to balance model capacity against computation time, with the learning rate range for XGBoost covering slow, moderate, and faster convergence schedules.

3.6 Evaluation Metrics

All tuned models were evaluated on the held-out test set using four metrics: accuracy, weighted F1-score, weighted precision, and weighted recall.

Accuracy

Accuracy alone is insufficient for this dataset because the classes are mildly imbalanced. A model that predicts the majority class more often would achieve artificially high accuracy without being genuinely useful across all gas types.

Weighted F1-Score

Weighted F1-score was chosen as the primary metric because it is the harmonic mean of precision and recall, and the weighted variant accounts for class imbalance by weighting each class's score by its support. This gives a single number that reflects performance across all six classes proportionally.

Precision and Recall

Precision measures what proportion of a model's positive predictions for a class were actually correct. Recall measures what proportion of actual instances of a class were correctly identified. Both are reported in their weighted form. In a gas detection context, both matter. A false positive could trigger an unnecessary alarm, while a false negative could mean failing to detect a dangerous gas.

A confusion matrix was also generated for the best performing model to provide a detailed breakdown of which gas types were misclassified and in what direction.

4. Results and Discussion

4.1 Model Performance Summary

Table 1 presents the performance of all six tuned models evaluated on the held out test set. All models achieved strong results on this dataset, with five of the six surpassing 98% test accuracy after tuning.

Model	Train Accuracy	Test Accuracy	Test F1 (weighted)	Test Precision	Test Recall
KNN	1.0000	0.9932	0.9932	0.9932	0.9932
Decision Tree	1.0000	0.9770	0.9770	0.9771	0.9770
SVM	0.9862	0.9853	0.9853	0.9853	0.9853
Logistic Regression	0.9145	0.9155	0.9151	0.9149	0.9155
Random Forest	1.0000	0.9914	0.9914	0.9914	0.9914
XGBoost	1.0000	0.9907	0.9907	0.9907	0.9907

Table 1: Model Performance on Held Out Test Set

As shown in Figure 8, KNN achieved the highest test accuracy at 99.32%, followed by Random Forest at 99.14% and XGBoost at 99.07%. Logistic Regression was clearly the weakest performer at 91.55%, while Decision Tree and SVM fell in the middle of the range. Table 2 presents the best hyperparameter combination selected by GridSearchCV for each model. These represent the configurations that achieved the highest average weighted F1-score across the five cross-validation folds on the training set.

Model	**Best Hyperparameters**
KNN	'metric': 'manhattan', 'n_neighbors': 3, 'weights': 'distance'}
Decision Tree	'criterion': 'entropy', 'max_depth': 20, 'min_samples_split': 2
SVM	'C': 10, 'gamma': 'scale', 'kernel': 'rbf'
Logistic Regression	'C': 10, 'max_iter': 500, 'solver': 'lbfgs'
Random Forest	'max_depth': 20, 'min_samples_split': 2, 'n_estimators': 300
XGBoost	'learning_rate': 0.2, 'max_depth': 6, 'n_estimators': 200

Table 2: Best Hyperparameters Selected by GridSearchCV

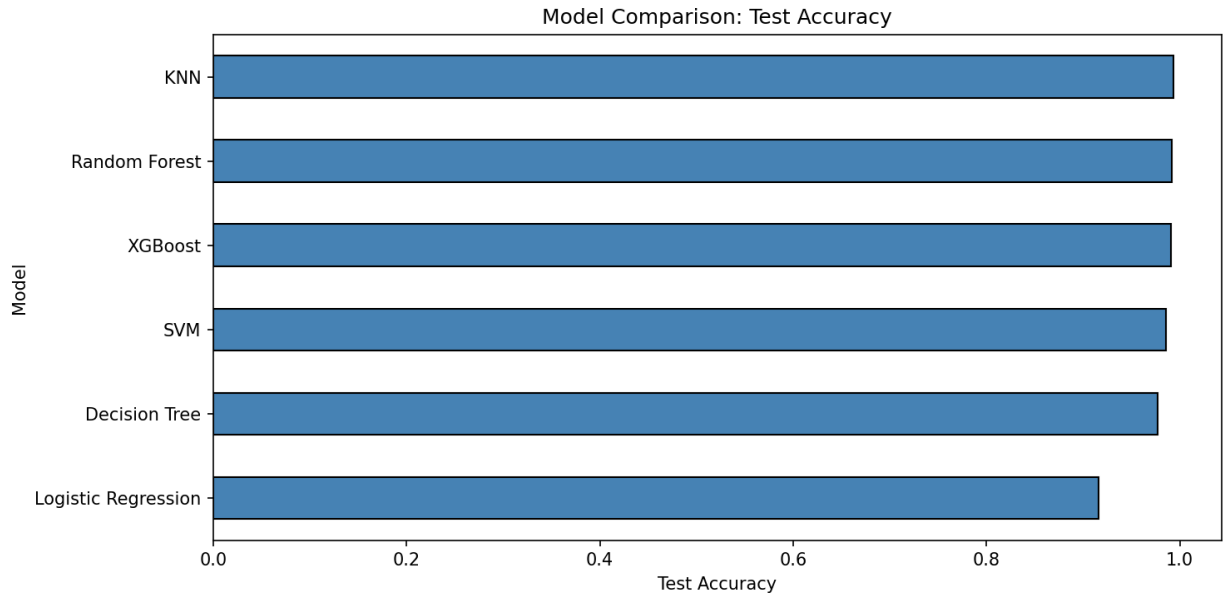

Figure 8: Model Test Comparison

4.2 Best Model Analysis: KNN

KNN was the best performing model with a test accuracy and weighted F1-score of 0.9932. The confusion matrix in Figure 9 provides a detailed breakdown of its predictions across all six gas classes.

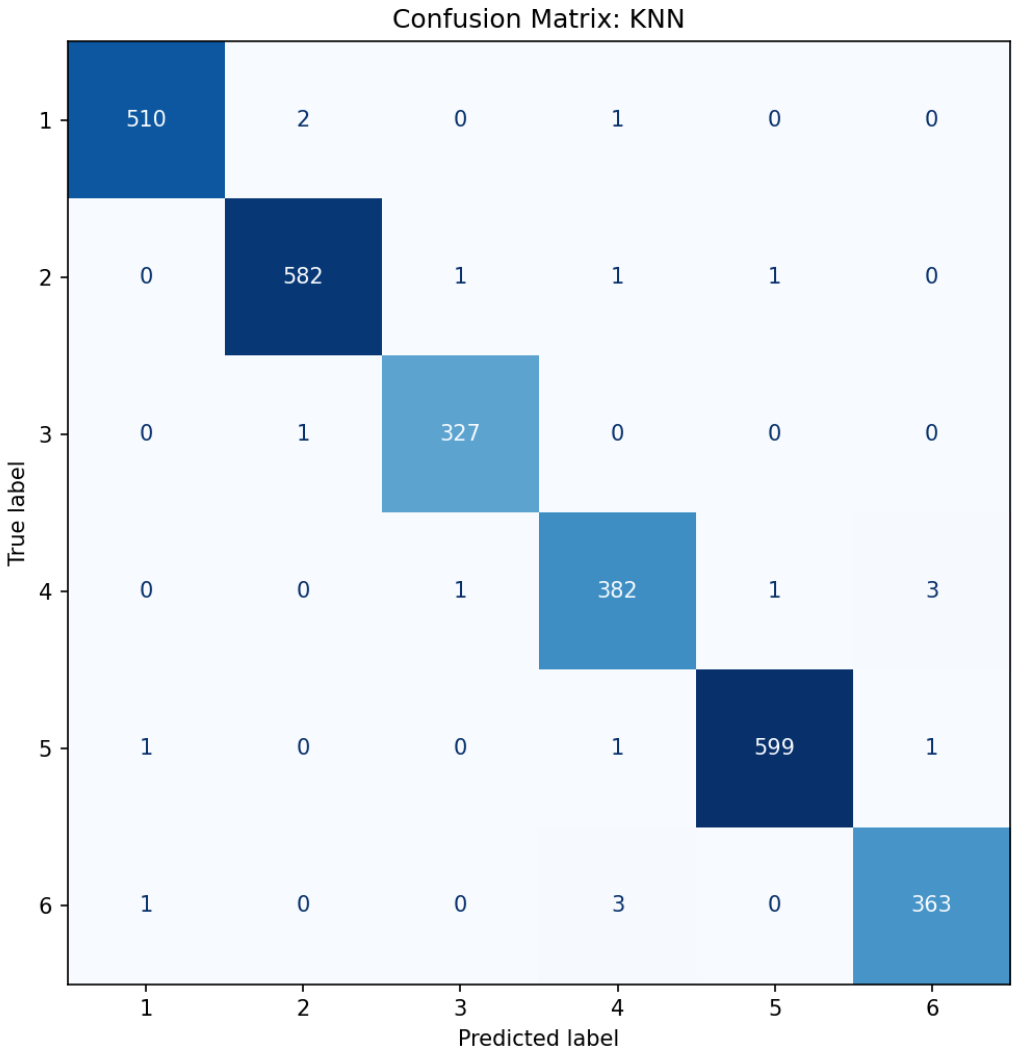

Figure 9: Confusion Matrix KNN

The diagonal entries: 510, 582, 327, 382, 599, and 363 represent the correct predictions for Ethanol, Ethylene, Ammonia, Acetaldehyde, Acetone, and Toluene respectively. The off diagonal values are remarkably sparse. The most notable misclassifications are 3 Acetaldehyde samples predicted as Toluene, and 3 Toluene samples predicted as Acetaldehyde. This bidirectional confusion between classes 4 and 6 is the clearest error pattern in the model. This suggests that Acetaldehyde and Toluene produce the most similar sensor response profiles of the six gases.

Overall, the confusion matrix confirms that KNN is highly reliable across all six classes with no systematic failure in any particular gas type. Table 3 presents the per class precision, recall, and F1-score from the full classification report for the KNN model, providing a more granular view of performance across individual gas types.

Class	Gas	Precision	Recall	F1-Score	Support
1	Ethanol	1.00	0.99	1.00	513
2	Ethylene	0.99	0.99	0.99	585
3	Ammonia	0.99	1.00	1.00	328
4	Acetaldehyde	0.98	0.99	0.99	387
5	Acetone	1.00	1.00	1.00	602
6	Toluene	0.99	0.99	0.99	367

Table 3: Per-Class Classification Report for KNN

The results are consistently strong across all six classes. Acetaldehyde and Toluene show lower F1-scores, which is consistent with the confusion matrix finding that these two classes are the most likely to be confused with each other. Ammonia, despite being the smallest class with only 328 test samples, achieved strong results, suggesting the model is not struggling with the mild class imbalance. Acetone achieved solid scores across the board, which is notable given that it is also the most represented class.

4.3 Feature Importance Analysis

Although KNN does not produce feature importances directly, the Random Forest model provides useful insight into which PCA components carried the most discriminative weight. As shown in Figure 10, PC2 was the most important component with an importance score of approximately 0.18, followed by PC6 at 0.147 and PC3 at 0.122. The remaining components contribute progressively less, with a gradual decline from PC10 onward.

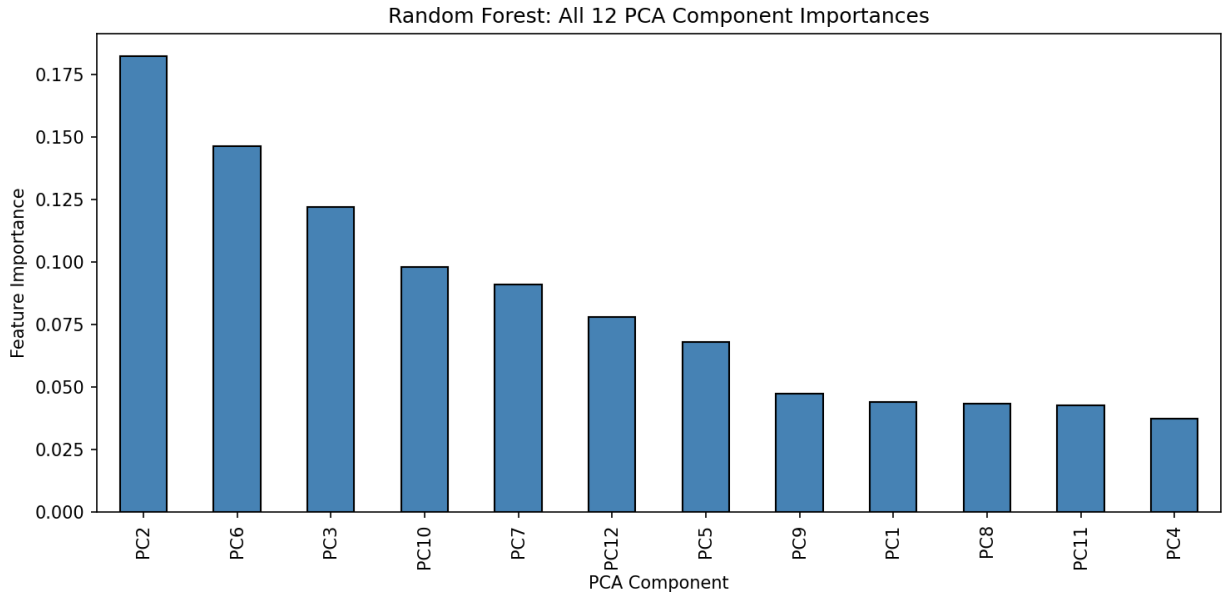

Figure 10: Feature Importance

The fact that importance is distributed across all 12 components (rather than concentrated in one or two) suggests that the gas classification problem requires information from multiple

directions in the transformed feature space. No single component is dominant enough to carry the task alone, which is consistent with the multi-sensor, multi-feature nature of the original data.

4.4 Discussion

Why did KNN perform the best?

KNN's strong performance on this dataset is consistent with what the EDA revealed. The box plots in section 2.5 showed that the different gas classes occupy visibly distinct regions of the feature space, particularly for features like feature_0 and feature_5. When classes are well separated in feature space, a distance based method like KNN is naturally well suited. It simply identifies the nearest labeled neighbors and assigns accordingly. After PCA reduced the noise and redundancy from 128 features down to 12 informative components, the class boundaries became even cleaner for KNN to take advantage of.

Why did ensemble methods perform slightly below KNN?

Random Forest and XGBoost both achieved nearly perfect results and outperformed every other model type. Their slightly lower performance to KNN is not a meaningful weakness. The difference is less than 0.3 percentage points. It is worth noting that both ensemble models showed a perfect training accuracy of 1.0000. This suggests a mild overfitting on the training data despite the regularization provided by their respective hyperparameter grids. KNN with distance weighting on the other hand adapts more smoothly to the local structure of the data without memorizing it the same way.

Why did the Decision Tree underperform relative to Random Forest?

The Decision Tree achieved 97.70% test score accuracy despite also achieving perfect training accuracy. A gap of 2.3 percentage points between train and test. This is essentially a textbook illustration of overfitting in a single tree. Random Forest mitigates this by averaging predictions across many trees trained on random subsets of the data, which reduces variance. The Decision Tree has no such averaging mechanism, making it more sensitive to the specific training samples it was exposed to.

Why did Logistic Regression struggle?

Logistic Regression's 91.55% test accuracy stands noticeably apart from the rest of the field. This is consistent with the nature of the model. Logistic Regression constructs linear decision boundaries in the feature space. Even after PCA, the gas classes are not linearly separable. The class separation visible in section 2.5 shows overlapping distributions and complex shapes that a linear model cannot fully capture. This result is not a failure of the model necessarily, but rather an appropriate illustration of the limits of linear classification on a non-linear problem.

Was the improvement from hyperparameter tuning significant?

The LazyPredict survey in Section 3.3 provided untuned baseline estimates. Comparing those to the final tuned results, the improvements were modest for the top performers. KNN, Random Forest, and XGBoost were already strong before tuning. The more meaningful gains came from the models in the middle of the range, where the choice of hyperparameters had a greater impact on performance. This is expected. When a model is already well suited to the data, tuning refines rather than completely transforming its performance.

4.5 Limitations

While the results of this project are strong, several limitations should be acknowledged.

First, this project treated the full 36-month dataset as a single static pool of samples, ignoring the temporal structure of the data. The dataset was collected in 10 distinct batches over time, and sensor drift means the statistical properties of the data shift between batches. By shuffling all samples together before splitting, the training set inevitably contains samples from later batches that would not be available in a real deployment scenario. This means the reported accuracy figures likely overestimate how well these models would perform on genuinely unseen future data.

Second, KNN has a notable practical limitation that is worth acknowledging despite its strong performance here. KNN is a lazy learner — it does not build an explicit model during training, but instead stores the entire training set and computes distances at prediction time. This means inference becomes increasingly slow and memory intensive as the training set grows. For a dataset of this size the impact is manageable, but KNN would not scale well to significantly larger datasets without approximation techniques.

Third, the hyperparameter search spaces used in this project were intentionally kept modest to ensure reasonable computation times. It is possible that wider search ranges or the use of RandomizedSearchCV over a larger space could yield marginally better results, particularly for models like SVM and XGBoost where the hyperparameter landscape is more complex.

Finally, the feature importance analysis in Section 4.3 was performed using Random Forest rather than the best performing model KNN, since KNN does not natively support feature importance extraction. While this provides useful directional insight, it reflects the importance structure learned by Random Forest specifically and may not perfectly represent how KNN is actually using the transformed feature space to make decisions.

5. Conclusion

5.1 Summary of Findings

This project applied six machine learning models to the Gas Sensor Array Drift Dataset with the goal of classifying six distinct gas types from 128-dimensional sensor array readings. Following a thorough exploratory data analysis, a preprocessing pipeline consisting of StandardScaler normalization and PCA dimensionality reduction was applied, reducing the feature space from 128 dimensions to 12 principal components while retaining 95% of the total variance. All six models were tuned using 5-fold cross-validated GridSearchCV and evaluated on a held-out test set using accuracy, weighted F1-Score, precision and recall.

The results were strong across the board. Five of the six models exceeded 98% test accuracy, and the best performing model (K-Nearest Neighbors) achieved a test accuracy and weighted F1-score of 99.32%. The dataset proved to be highly separable, with well defined class boundaries that favored distance based and ensemble methods over linear classifiers.

5.2 Research Questions Revisited

Which machine learning algorithm achieves the highest classification accuracy on the Gas Sensor Array Drift Dataset?

KNN achieved the highest test accuracy at 99.32%, edging out Random Forest at 99.14% and XGBoost at 99.07%. This outcome was somewhat surprising. Ensemble methods are often expected to outperform simpler approaches. However, the well separated class structure of this dataset, combined with the noise reduction provided by PCA, created conditions where KNN's distance based approach was particularly effective.

How do simpler models such as Logistic Regression and Decision Trees compare to ensemble methods such as Random Forest and XGBoost?

The gap between simpler and ensemble models was meaningful, but not dramatic. Logistic Regression was the clear outlier at a mere 91.55%, confirming that the gas classes are not linearly separable even in the PCA reduced space. The Decision Tree performed reasonably at 97.70% but showed a clear overfitting pattern; perfect training accuracy with a 2.3 percentage point drop on the test set. Random Forest and XGBoost, by contrast demonstrated that the ensemble methods reliably close this gap through variance reduction, making them the most dependable choices when consistent generalization is the priority.

Does dimensionality reduction via PCA meaningfully reduce the feature space while preserving classification performance?

Yes, PCA reduced the feature space from 128 dimensions to just 12 components. A reduction of over 90% while retaining 95% of the total variance. The higher inter feature

correlation identified in the EDA, which is expected given the multi sensor structure of the dataset, made this compression possible without meaningful loss. All six models were trained on the PCA-reduced data and still achieved strong results, suggesting that the 12 components captured the essential discriminative information from the original features.

5.3 Future Work

While the results of this project are strong, several directions for future work could further advance the understanding of this dataset and improve upon the current approach.

First, this project treated the dataset as a static classification problem, ignoring the temporal structure of the data. The dataset was collected over 36 months across 10 batches, and sensor drift means that the statistical properties of the data change over time. A promising direction could be to train models on earlier batches and evaluate them on later batches, simulating a real-world deployment scenario. This would reveal how quickly model performance degrades over time and motivate the use of drift-compensation techniques such as domain adaptation or incremental learning.

Second, the feature engineering in this project relied entirely on the 128 pre-engineered features provided by the dataset. Future work could explore whether alternative feature representation, such as raw time series features extracted directly from the sensor response curves, or learned representations from autoencoders, any of which could improve classification performance or better capture the discriminative structure of the data.

References

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. https://doi.org/10.1145/2939672.2939785

Fine, G. F., Cavanagh, L. M., Afonja, A., & Binions, R. (2010). Metal oxide semi-conductor gas sensors in environmental monitoring. *Sensors (Basel, Switzerland)*. https://pmc.ncbi.nlm.nih.gov/articles/PMC3247717/

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference, 445*, 51–56. https://doi.org/10.25080/Majora-92bf1922-00a

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Vergara, A. (2012). Gas Sensor Array Drift Dataset [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5RP6W